Shri Dadaji Institute of Technology & Science, Khandwa

Department of Computer Science & Engineering

A

Practical file on

"Software Architecture"

Submitted to: Mr. Rajesh Gyanchandani **Submitted by:**

Department of CSE

Table Of Contents

Sr. No.	Title	Date of submission	Sign	Remark
1.	Study of software development life cycle.			
2	Study of UML diagrams, use, types and procedures			
3	Software Quality Assurance (QA) & Quality Control (QC)			
4	Study of different software development models			
5	Study of various software testing techniques			
6	Study of various software architecture model			

Practical-1

Aim- Study of software development life cycle

Theory:

Introduction

The Software Development Life Cycle (SDLC) is a structured process used by software developers to design, develop, test, and deliver high-quality software. It provides a systematic approach that helps ensure the final product meets client requirements, is reliable, and is delivered on time. SDLC also helps reduce project risks, manage costs, and maintain a clear workflow for both developers and clients.

Objectives of SDLC

- To provide a clear framework for software development
- To ensure the project is completed within time and budget
- To improve software quality through proper planning and testing
- To reduce risks and errors during development
- To ensure smooth communication among team members and stakeholders

Phases of SDLC

1. Requirement Analysis

In this phase, developers meet with clients or users to understand what the software needs to do. All functional and non-functional requirements are collected. This helps create a clear understanding of the project scope.

2. System Design

Based on the collected requirements, a system design is prepared. This includes architecture, data flow, user interface design, database design, and technology selection. The design acts as a blueprint for developers.

3. Implementation (Coding)

In this step, the actual code is written according to the design. Developers break the project into modules and write code using suitable programming languages. This is usually the longest phase.

4. Testing

Testing ensures that the software works correctly and is free of errors. Various tests such as unit testing, integration testing, system testing, and user acceptance testing (UAT) are performed.

5. Deployment

After successful testing, the software is delivered to the client or deployed to a live environment. It may be released in stages depending on the project plan.

6. Maintenance

Even after delivery, the software requires updates, bug fixes, and improvements. This phase ensures the software continues to function smoothly and meets future needs.

SDLC Models

Different SDLC models are used based on project needs:

1. Waterfall Model

A linear model where each phase is completed before moving to the next. It is simple but not suitable for changing requirements.

2. Agile Model

An iterative and flexible model where software is developed in small parts called sprints. It allows continuous feedback and frequent updates.

3. Spiral Model

Combines design and prototyping with risk analysis. Useful for large and high-risk projects.

4. V-Model

Also called the Verification and Validation model. Each development phase has a corresponding testing phase.

Advantages of SDLC

- Better project planning and management
- Improved software quality
- Clear documentation
- Reduced risks and development costs
- Higher customer satisfaction

Conclusion

The Software Development Life Cycle (SDLC) is essential for building reliable and efficient software. By following its structured phases—planning, designing, coding, testing, deployment, and maintenance—teams can minimize errors and ensure that the final product meets user expectations. SDLC provides a disciplined approach that helps deliver successful software projects.

Practical-2

Aim: Study of UML diagrams, use, types and procedures

Theory:

Introduction-:

Unified Modeling Language (UML) is a standardized visual modeling language used to design, understand, and document the structure and behaviour of software systems. UML helps developers, analysts, and stakeholders visualize a system before actual coding begins. It supports object-oriented concepts and provides multiple diagrams for better analysis and design.

Uses of UML

- Helps in planning and designing software systems
- Provides a clear visual representation of system architecture
- Improves communication between developers, clients, and designers
- Helps in identifying errors early in the development process
- Supports object-oriented modeling (classes, objects, interactions)
- Useful for documentation and future software maintenance

Procedures for Creating UML Diagrams

1. **Understand Requirements**
   Gather functional and non-functional requirements from stakeholders.
2. **Identify Key Elements**
   Identify actors, classes, objects, modules, activities, and states.
3. **Choose the Correct Diagram Type**
   Select structural diagrams for static design and behavioral diagrams for processes and interactions.
4. **Define Relationships**
   Add associations, dependencies, generalizations, and interactions.
5. **Create the Diagram**
   Using UML symbols, create the correct model with clarity and consistency.

6. **Review and Validate**
   Verify the diagram with developers and clients to ensure accuracy.

Types of UML Diagrams

UML diagrams are mainly divided into **two types: Structural and Behavioural**. Structural diagrams show the *static* design of a system, while behavioural diagrams show the *dynamic* interactions and workflows within the system.

A) Structural UML Diagrams

These diagrams show how different parts of a system are organized and related.

1. Class Diagram

- Shows classes, attributes, and methods
- Represents relationships like inheritance and association
- Used for object-oriented design

2. Object Diagram

- Shows objects and their states at a specific time
- Represents a snapshot of the system

3. Component Diagram

- Represents software components or modules
- Shows how components are connected

4. Deployment Diagram

- Shows hardware devices and how software is deployed on them
- Used for system architecture

5. Package Diagram

- Groups related classes and components into packages
- Helps organize large systems

6. Composite Structure Diagram

- Shows internal structure of a class

- Used for detailed class-level modeling

B) Behavioral UML Diagrams

These diagrams represent how the system behaves and interacts during execution.

1. Use Case Diagram

- Shows interactions between users and the system
- Helps collect system requirements

2. Sequence Diagram

- Shows time-based interaction between objects
- Represents message flow in order

3. Activity Diagram

- Shows workflows and processes
- Similar to a flowchart

4. State Machine Diagram

- Shows different states of an object
- Represents transitions caused by events

5. Communication Diagram

- Shows object interactions
- Focuses on structural relationships

6. Interaction Overview Diagram

- High-level interaction flow
- Combines elements of activity and sequence diagrams

C) Tools Used for Creating UML Diagrams

1. StarUML

A powerful UML modeling tool commonly used by students and professionals.

2. Lucidchart

An online diagramming tool with easy drag-and-drop features.

3. Draw.io (Diagrams.net)

A free browser-based tool for making UML and flow diagrams.

4. Visual Paradigm

Supports all UML diagrams with team collaboration features.

5. Microsoft Visio

A professional diagramming tool used in organizations.

6. IBM Rational Rose

A traditional enterprise-level UML designing tool.

7. Creately

Online tool with UML templates and collaboration options.

Conclusion

UML diagrams play a major role in software analysis and design. By using structural and behavioural diagrams, developers can clearly visualize system components, interactions, and workflows. UML improves communication, reduces errors, and supports better planning and documentation. With various tools available, creating UML diagrams becomes easier and more efficient.

Practical- 3

Aim: Software Quality Assurance (QA) & Quality Control (QC)

Theory:

Introduction: -

In software engineering, delivering a high-quality product is extremely important. To ensure this, organizations use two major practices: Software Quality Assurance (QA) and Software Quality Control (QC). Both focus on achieving quality but in different stages and with different approaches. QA ensures that the development process is correct, while QC ensures that the final software product meets the required standards.

Software Quality Assurance (QA)

Definition

Software Quality Assurance (QA) is a **process-oriented** approach that ensures the entire software development process follows proper standards, guidelines, and best practices. It aims to **prevent defects** by improving the development and design process. **Detailed Explanation**

QA focuses on establishing a strong development process so that fewer errors are introduced during coding or design. It involves monitoring and improving the SDLC processes, conducting audits, reviewing documentation, and ensuring compliance with quality standards such as ISO or CMMI.

Key Features / Points

- QA works **before and during** development.
- Ensures all team members follow defined processes.
- Helps reduce defects early, saving time and cost.
- Includes **process audits, reviews, walkthroughs**, and quality planning.
- Proactive approach: stops problems before they occur.

Software Quality Control (QC)

Definition

Software Quality Control (QC) is a **product-oriented** activity that focuses on testing the final software to identify and fix defects. Its goal is to ensure that the software works correctly and meets user expectations.

Detailed Explanation

QC is performed after the software has been developed. It involves executing the application, testing its features, and finding bugs. The QC team uses test cases, testing tools, and bugreporting systems to check the software's performance, reliability, usability, and functionality.

Key Features / Points

- QC works **after** development is completed.
- Focuses on actual product testing.
- Helps find and fix defects before software release.
- Includes **unit testing, integration testing, system testing, and acceptance testing**.
- Reactive approach: detects problems after they occur.
- Ensures the final software meets customer requirements.

Conclusion

Software QA and QC work together to deliver a high-quality software product. QA improves the **process** to avoid defects, while QC tests the **product** to find existing defects. Both are essential for building reliable, efficient, secure, and user-friendly software. Without QA and QC, software projects may face issues such as failures, low performance, security risks, and customer dissatisfaction.

Practical- 4

Aim: Study of different software development models **Theory:**

Introduction

Software development models are structured approaches used to plan, design, build, test, and deliver software. Each model follows a unique process and is chosen based on project size, complexity, requirements, and timelines. These models help developers organize their work, reduce errors, and deliver high-quality software efficiently.

1. Waterfall Model

The Waterfall Model is one of the earliest and simplest software development models. It follows a **linear and sequential** approach, where each phase must be completed before moving to the next.

Key Features

- Highly structured and easy to understand
- Each phase has clear documentation
- Best suited for projects with fixed and stable requirements

Phases

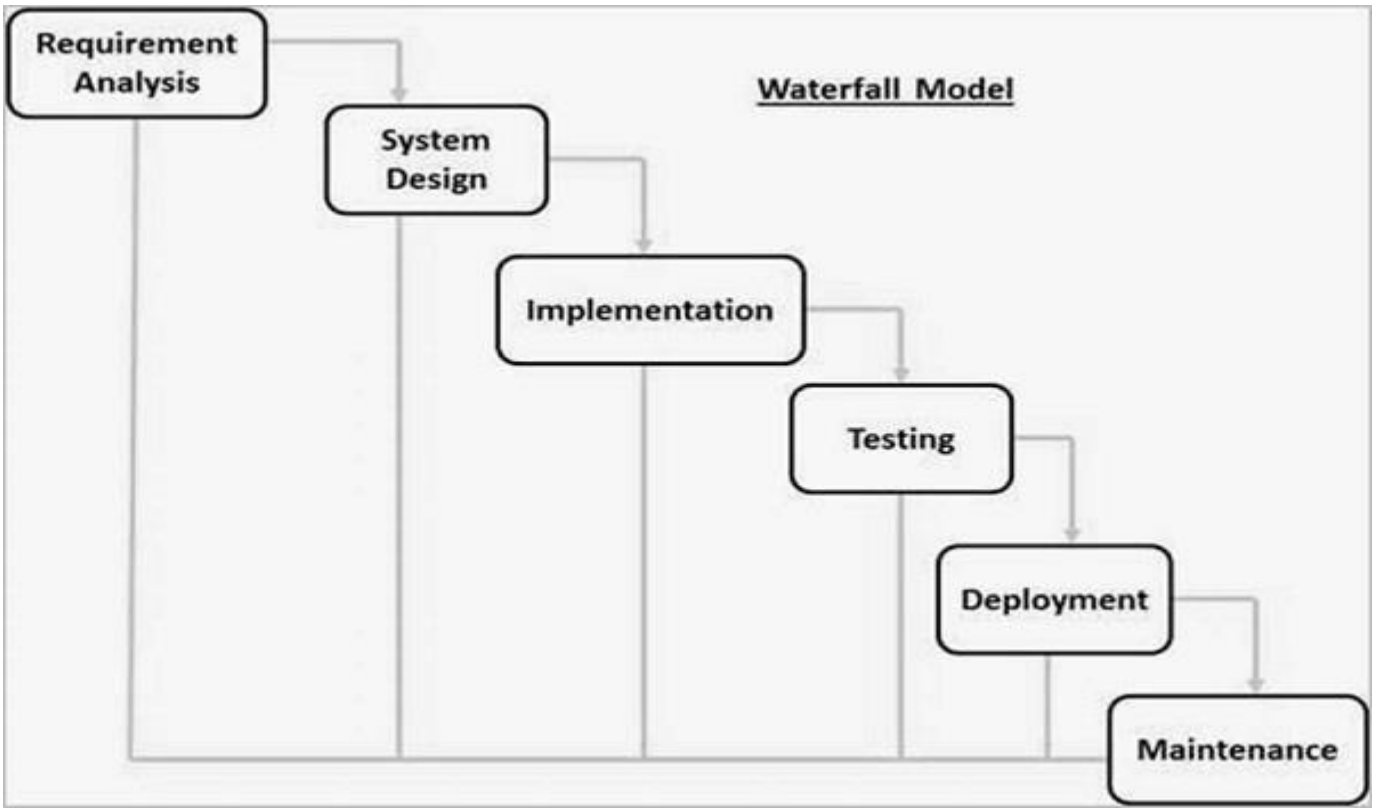
1. Requirement Analysis
2. System Design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

Advantages

- Simple and easy to manage
- Good for small projects

Disadvantages

- Not flexible
- Changes are difficult once a phase is completed



2. Agile Model

Agile Model is an iterative and incremental approach. The project is divided into small cycles called sprints, and working software is delivered after each sprint.

Key Features

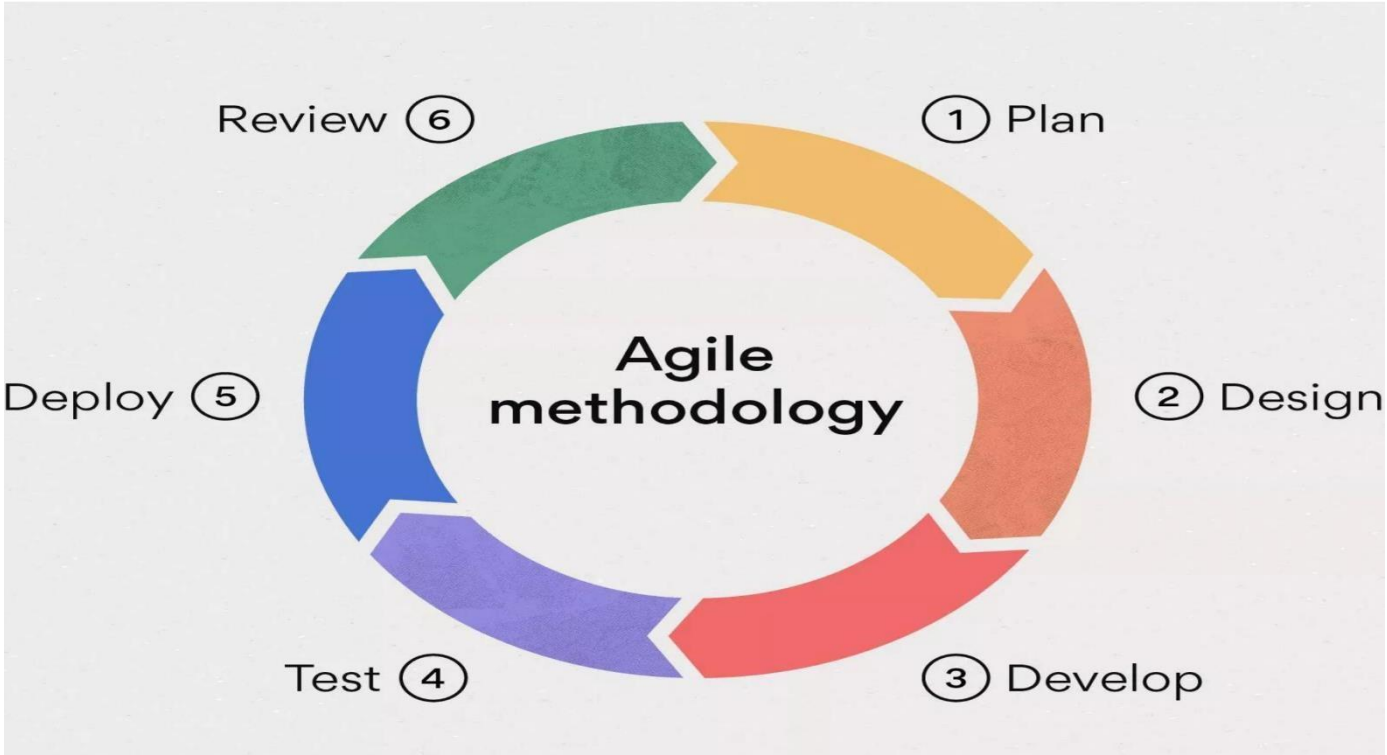
- Focuses on customer feedback
- Highly flexible and adaptive

Advantages

- Quick delivery of working software
- Easy to handle changing requirements

Disadvantages

- Requires experienced team
- May lack proper documentation



3. Spiral Model

Spiral Model combines elements of both Waterfall and Iterative models. It is built around **risk analysis** and is best for large, high-risk projects.

Key Features

- Project is developed in four repeating phases: Planning, Risk Analysis, Development, Evaluation

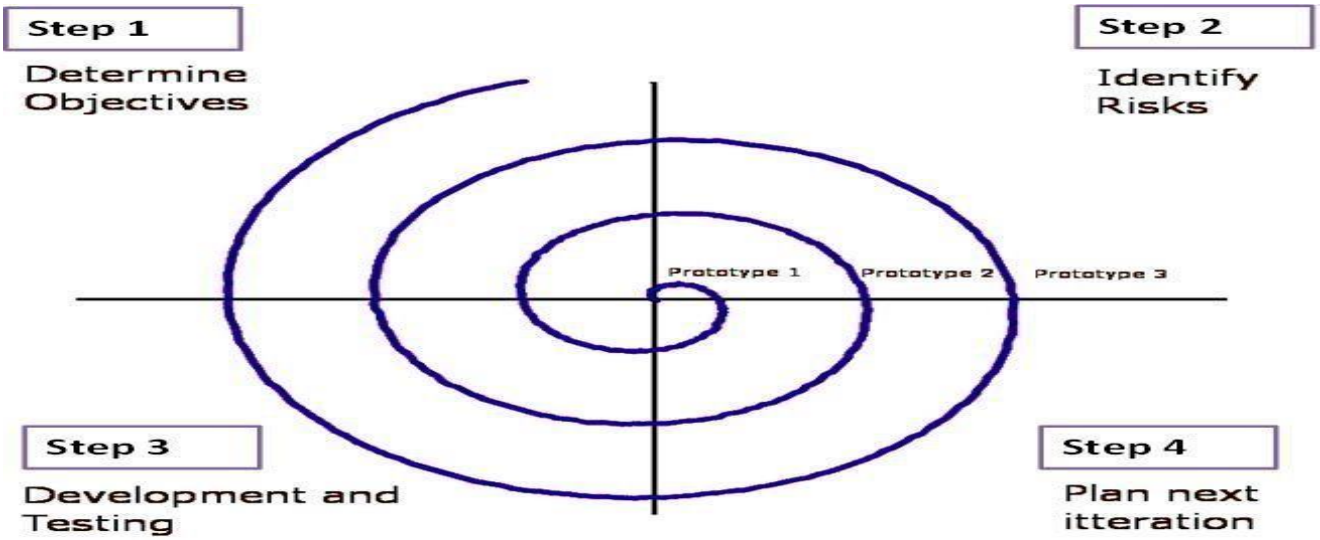
- Greater focus on managing risks

Advantages

- Ideal for complex and risky projects
- Allows continuous refinement

Disadvantages

- Expensive and time-consuming
- Requires expertise in risk management



5. Iterative Model

In this model, the development is done in **small repeated cycles**. Each version (iteration) of the software adds new features.

Key Features

- Useful when requirements are not fully known
- Each iteration improves the previous version

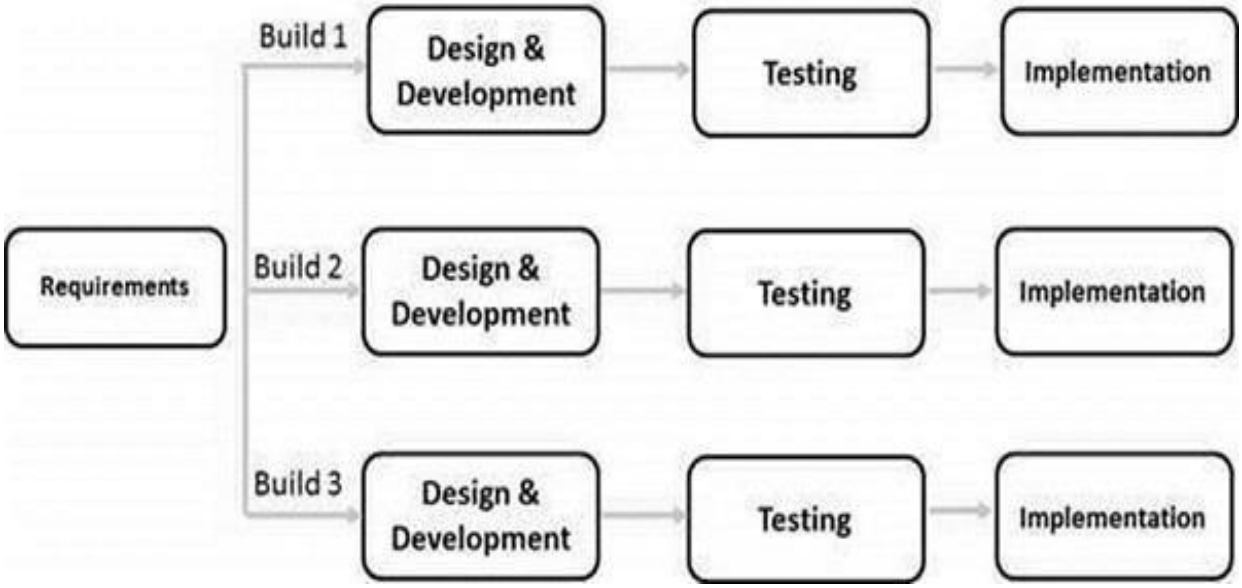
Advantages

- Easy to manage changes

- Early working model available

Disadvantages

- Requires good planning
- Can be costly



6. Prototype Model

This model involves building a prototype early to understand user requirements. Once the user approves the prototype, actual development begins.

Key Features

- Helps clarify unclear requirements
- User actively involved

Advantages

- Reduces misunderstandings
- Improves user satisfaction

Disadvantages

- Prototype may increase time and cost
- Users may confuse prototype with final product

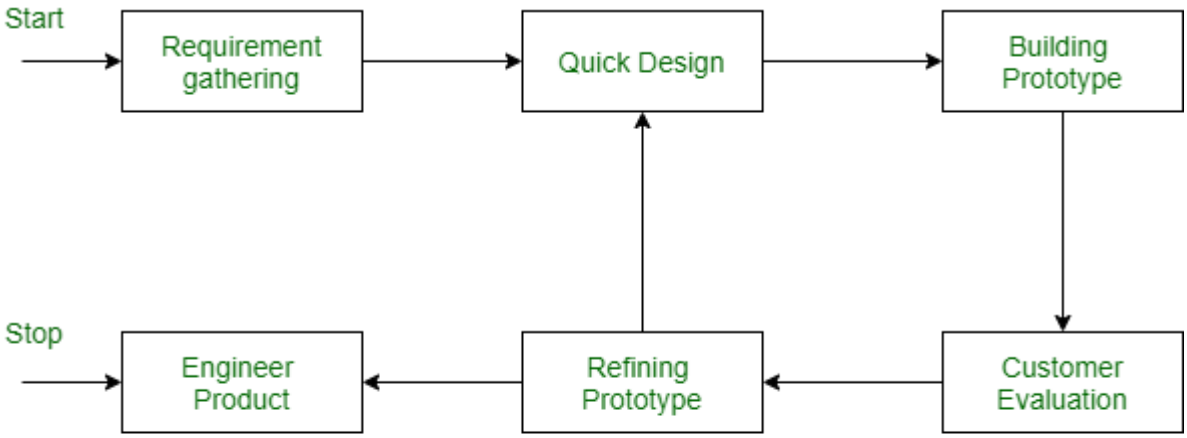


Figure - Prototype Model

Conclusion

Different software development models serve different project needs. Waterfall works well for simple, fixed projects; Agile fits dynamic requirements; Spiral is ideal for large, risky projects; and models like Prototype and Iterative help refine unclear requirements. Choosing the right model improves project success, quality, and timely delivery

Practical 5

Aim: Study of various software testing techniques **Theory:**

Introduction

Software testing techniques are systematic methods used to identify defects, ensure software quality, and verify that the system meets user requirements. These techniques help testers check functionality, performance, reliability, and security. Testing techniques are mainly divided into Black-box, White-box, and Grey-box approaches, each suited for different testing needs.

1. Black-Box Testing Techniques

Black-box testing focuses on testing the **functionality** of the software without looking at the internal code. Testers only check inputs and outputs.

a) Equivalence Partitioning

- Divides input data into valid and invalid partitions
- Test cases are selected from each partition
- Reduces the number of test cases while ensuring coverage

b) Boundary Value Analysis

- Tests values at the boundaries (e.g., minimum, maximum)
- Effective for catching edge-case errors • Commonly used for input field testing

c) Decision Table Testing

- Represents complex business logic using tables
- Useful when multiple conditions affect outcomes

- Ensures all combinations are tested

d) State Transition Testing

- Used when the system changes behavior based on state
- Tests valid and invalid state transitions
- Useful for login systems, ATM machines, etc.

e) Use Case Testing

- Tests the system based on real user scenarios
- Ensures end-to-end functionality works

2. White-Box Testing Techniques

White-box testing focuses on the **internal structure, logic, and code** of the software. Testers must know programming.

a) Statement Coverage

- Ensures every line of code is executed at least once •

  Helps identify unused code or dead code

b) Branch Coverage

- Tests all possible decision paths (if/else, loops) •

  Ensures each branch is executed

c) Condition Coverage

- Tests all Boolean conditions independently •

  Gives deeper logical coverage

d) Path Coverage

-
- Tests all possible paths in the code

  Ensures full flow coverage, though complex for large systems

3. Grey-Box Testing Techniques

Grey-box testing combines both Black-box and White-box techniques. Testers know partial internal details.

a) Matrix Testing

- Uses requirement and risk matrices •

  Helps find weak areas in the system

b) Regression Testing

- Ensures changes or fixes do not break existing functionality •

  Grey-box knowledge helps select impacted areas

c) Pattern-Based Testing

- Tests based on known system patterns and structures
- Useful for applications with similar modules

4. Static Testing Techniques

Static testing is performed **without executing the code**.

a) Reviews

- Team members review documents, code, and design •

  Helps identify errors early

b) Walkthroughs

-
- Presenting the design/code to a group for feedback •

  Conducted informally

c) Code Inspection

Formal review of code to detect logical or structural errors

- Very effective in improving code quality

5. Dynamic Testing Techniques

Dynamic testing involves executing the software to find defects.

a) Functional Testing

- Tests system functions according to requirements
- Includes unit, integration, system, and acceptance testing

b) Non-Functional Testing

Checks performance, usability, security, etc.
Examples:

- Load Testing
- Stress Testing
- Security Testing
- Compatibility Testing
- Usability Testing

Conclusion

Different software testing techniques help ensure that the software is reliable, functional, and error-free. Black-box techniques validate external behavior, white-box

•

techniques test internal logic, and grey-box techniques combine both. Static techniques help detect issues early, while dynamic techniques verify actual performance. Using the right combination of these techniques results in higher-quality software and better user satisfaction

Practical 6

Aim: Study of various software architecture model **Theory:**

Introduction

Software architecture models describe the high-level structure of a software system. They define how components interact, how data flows, and how the system is organized. Using the right architecture model helps ensure scalability, security, performance, and easier maintenance. Different projects require different architectural approaches depending on size, complexity, and requirements.

1. Layered Architecture (N-Tier Architecture)

One of the most common and traditional architectures. The system is divided into layers where each layer handles a specific responsibility.

Layers

- Presentation Layer – User interface
- Business Layer – Business logic
- Data Access Layer – Data handling
- Database Layer – Storage system

Features

- Easy to develop and maintain
- High separation of concerns
- Ideal for enterprise applications

Examples

Banking systems, ERPs, school management systems

2. Client–Server Architecture

In this model, the application is divided into two parts: a **client** (front-end) and a **server** (back-end).

Features

- Client sends requests; server processes and returns responses
- Can support multiple clients
- Useful for centralized databases

Examples

Web applications, online banking, email systems

3. MVC Architecture (Model–View–Controller)

A popular architecture used for web and mobile applications.

Components

- **Model** – Data and business logic
- **View** – User interface
- **Controller** – Handles user input and communication

Features

- Clear separation of UI and logic
- Easy to maintain and test
- Supports parallel development

Examples

React, Angular, Django, Laravel, Spring MVC

4. Microservices Architecture

A modern architecture where applications are broken into small, independent services.

Features

- Each service performs a single function
- Services communicate via APIs
- Supports independent deployment

Advantages

- Highly scalable
- Easy to update service-wise

Examples

Netflix, Amazon, Uber, large distributed systems

5. Event-Driven Architecture

In this model, the system reacts to events such as user actions or messages.

Features

- Loose coupling between components
- Handles real-time data efficiently
- Suitable for highly dynamic applications

Examples

Sensor systems, trading applications, real-time analytics

6. Pipe-and-Filter Architecture

Data passes through a series of processing elements called filters.

Features

- Easier to modify or add filters
- Good for data transformations

Examples

Compilers, data processing systems, streaming systems

Conclusion

Software architecture models play a crucial role in defining how a system is structured, how it behaves, and how easily it can be maintained or scaled. Traditional models like Layered and Client-Server support simple systems, while modern approaches like Microservices and Event-Driven architectures support large, complex, and scalable applications. Choosing the right architecture ensures better performance, flexibility, and long-term reliability of the software system.